Mobile application Assignement

Question 1

1.1 Step-by-Step Process of Creating a Simple Android Application

Creating an Android application involves several sequential steps, from project setup to deployment. Below is a comprehensive step-by-step guide:

Step 1: Install and Set Up Android Studio

- Download and install **Android Studio**, the official Integrated Development Environment (IDE) for Android development.
- Configure the **Android SDK**, emulator, and necessary plugins during the installation process.
- Android Studio provides templates and tools to simplify development (Android Developers, 2023).

Step 2: Create a New Project

- Open Android Studio and click **“New Project”**.
- Choose a project template (e.g., **Empty Activity**) suitable for your application.
- Enter project details:
    - **Application name** – the name of your app.
    - **Package name** – unique identifier for your app.
    - **Save location** – folder path for project files.
    - **Language** – Java or Kotlin.
    - **Minimum API level** – defines the oldest Android version supported.
- Click **Finish** to generate the project structure.

Step 3: Design the Layout

- Open activity_main.xml in **res/layout**. This file uses XML to define the app’s user interface.
- Drag and drop UI components such as TextView, Button, and EditText from the **Palette**, or code directly in XML.
- Example layout XML snippet:

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

android:orientation="vertical"

```
android:layout_width="match_parent"

android:layout_height="match_parent"

android:padding="16dp">

<TextView

android:id="@+id/tvWelcome"

android:text="Welcome to My App"

android:textSize="20sp"

android:layout_width="wrap_content"

android:layout_height="wrap_content" />

<Button

android:id="@+id/btnClick"

android:text="Click Me"

android:layout_width="wrap_content"

android:layout_height="wrap_content" />

</LinearLayout>
```

- The layout defines **how the user interface looks and behaves** (Android Developers, 2023).

Step 4: Implement Activity

- Open MainActivity.java (or MainActivity.kt for Kotlin).
- Activities control the behavior of UI components.
- Example Java implementation:

```
package com.example.myapp;

import android.os.Bundle;

import android.widget.Button;

import android.widget.TextView;
```

```
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

@Override

protected void onCreate(Bundle savedInstanceState) {

super.onCreate(savedInstanceState);

setContentView(R.layout.activity_main);

TextView tvWelcome = findViewById(R.id.tvWelcome);

Button btnClick = findViewById(R.id.btnClick);

btnClick.setOnClickListener(v -> tvWelcome.setText("Button Clicked!"));

}

}
```

Step 5: Testing

- Use **Android Emulator** or a **physical device** to run the application.
- Click **Run → Run 'app'** in Android Studio.
- Test user interactions, UI behavior, and app functionality to ensure stability (Burnette, 2018).

Step 6: Debugging

- Use **Logcat** to monitor runtime logs.
- Fix errors such as null pointer exceptions, UI misalignment, or crashes.
- Testing ensures that the app is reliable before release.

Step 7: Deployment

- Generate a signed APK or **App Bundle** for distribution:
  - **Build → Generate Signed Bundle / APK** in Android Studio.
- Upload to **Google Play Store** or distribute directly.
- Deployment requires **code signing** with a private key to ensure authenticity (Android Developers, 2023).

References for Step 1.1:

- Android Developers (2023). *Get started with Android Studio*. [online] Available at: https://developer.android.com/studio [Accessed 19 Sep. 2025].
- Burnette, E. (2018). *Hello, Android: Introducing Google's Mobile Development Platform*. 5th ed. San Francisco: Pragmatic Bookshelf.

1.2 Role of XML in Android Application Development

XML (Extensible Markup Language) plays a critical role in **defining user interfaces and app resources** in Android development. Its main functions include:

1. Designing User Interfaces

- XML files in res/layout define the arrangement and appearance of UI elements.
- Components such as TextView, Button, and RecyclerView are specified in XML.
- Example (activity_main.xml):

<Button

android:id="@+id/btnSubmit"

android:text="Submit"

android:layout_width="wrap_content"

android:layout_height="wrap_content" />

2. Defining Layouts

- XML allows defining **layout hierarchies** like LinearLayout, RelativeLayout, or ConstraintLayout.
- Layout XML files manage alignment, padding, margins, and orientation without writing Java/Kotlin code (Android Developers, 2023).

3. Configuring App Resources

- XML files also define **colors, strings, and themes**:
    - res/values/colors.xml – defines color palette:

<color name="primaryColor">#6200EE</color>

- res/values/strings.xml – stores all text values:

<string name="app_name">My App</string>

- res/values/styles.xml – defines themes and styles:

<style name="AppTheme" parent="Theme.MaterialComponents.DayNight">

<item name="colorPrimary">@color/primaryColor</item>

<item name="colorAccent">@color/teal_200</item>

</style>

Other Common XML Files

1. **AndroidManifest.xml** – declares app components (activities, services, permissions) and app metadata.
2. **layout XML files** – define UI for each screen/activity.
3. **values XML files** – include strings, colors, dimensions, and styles for consistent UI.

References for Step 1.2:

- Android Developers (2023). *App resources overview*. [online] Available at: https://developer.android.com/guide/topics/resources/overview [Accessed 19 Sep. 2025].
- Burnette, E. (2018). *Hello, Android*. 5th ed. San Francisco: Pragmatic Bookshelf.